

数据结构与算法

1. 数据结构

逻辑结构：指数据元素之间的逻辑关系，分为线性结构和非线性结构，线性结构包含线性表、栈和队列、串等数据结构，非线性结构包含集合、树、图等数据结构。

存储结构：指数据结构在计算机中的表示，主要有顺序存储、链式存储、索引存储、散列存储。

2. 复杂度

时间复杂度：通常使用大O表示法， $T(n) = O(f(n))$ ，其中 $f(n)$ 表示每行代码执行次数之和， O 表示正比例关系。表示算法执行时间随数据规模增长的变化趋势，只考虑算法的主要操作数量级，忽略常数因子和低阶项。

空间复杂度：通常使用大O表示法，表示算法所需的额外内存空间随数据规模增长的变化趋势。

3. P/NP问题

P问题：可以在多项式时间内解决的问题

NP问题：可以在多项式时间内验证一个解的正确性，但不一定能够在多项式时间内找到这个解

4. 排序算法

冒泡排序：从序列的第一个元素开始，比较 **相邻** 的两个元素，如果它们的顺序错误，则交换它们的位置，直到最终完成排序。

- 最好情况：待排序序列有序，只需要扫描1趟，不需要进行交换操作
- 最坏情况：待排序序列逆序，需要扫描 $n-1$ 趟，交换 $n(n-1)/2$ 次
- 优化：如果某趟排序没有发生交换，则可以提前结束排序

选择排序：将序列分为有序部分和无序部分，每次在无序部分找到最小值，与无序部分的第一个元素交换位置。

- 优化：在一趟遍历中，同时找出最大值与最小值，放到序列两端

插入排序：将序列分为有序部分和无序部分，从无序部分依次选择元素与有序部分比较，找到合适的位置，将原来的元素往后移，将元素插入到相应位置上。

- 最好情况：待排序序列有序，需要比较 $n-1$ 次
- 最坏情况：待排序序列逆序，需要比较 $n(n-1)/2$ 次
- 优化：折半插入排序、希尔排序

希尔排序：先将序列分为若干个子序列，每个子序列的元素之间相隔某个 **增量**，对各个子序列进行直接插入排序，等到序列基本有序时，再对整个序列进行一次直接插入排序。

堆排序：将序列中的元素构建成一个堆，不断移除堆顶元素，并将剩余元素重新调整为堆，直到堆为空。

归并排序：把两个或两个以上的有序子序列合并成一个新的有序序列。

快速排序：在序列中选取一个 **基准元素**，将待排序的序列分成两部分，前半部分均小于基准元素，后半部分均大于或等于基准元素，然后分别对前后两部分重复上述操作，直到最终完成排序。

排序方法	平均情况	最好情况	最坏情况	辅助空间	稳定性
冒泡排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	稳定
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
直接插入排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	稳定
希尔排序	$O(n \log n) \sim O(n^2)$	$O(n^{1.3})$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n) \sim O(n)$	不稳定

5. 查找算法

顺序查找：逐个检查序列中的元素，直到找到目标元素或搜索完所有元素。

- 时间复杂度： $O(n)$

二分查找：在 **有序** 序列中，将目标值与中间元素进行比较，如果目标值等于中间元素，则搜索过程结束；如果目标值大于或小于中间元素，则在序列大于或小于中间元素的那一半中继续查找，直到找到目标元素或搜索范围为空。

- 时间复杂度： $O(\log_2 n)$

二叉搜索树：若左子树不空，则左子树上所有结点的值都小于根结点的值；若右子树不空，则右子树上所有结点的值都大于根结点的值；任意结点的子树也都是二叉搜索树。

- 时间复杂度： $O(\log_2 n) \sim O(n)$

平衡二叉树：平衡的二叉搜索树，即左子树和右子树的高度差的绝对值不超过1，且左子树和右子树都是平衡二叉树。

- 时间复杂度： $O(\log_2 n)$
- 解决了输入数据接近有序时，树的结构形似链表的问题

B树：平衡的 **m路** 搜索树，满足每个结点至多有m个子结点，每个非根非叶子结点至少有 $\lceil m/2 \rceil$ 个子结点；如果根结点不是叶子结点，那么它至少有两个子结点；非叶子结点的关键字个数=子结点个数-1，且升序排列；所有的叶子结点都在同一层，且不含有任何信息。

- 时间复杂度： $O(\log_m n)$
- 解决了数据量太大而造成树的高度过高的问题

B+树：平衡的 **m路** 搜索树，满足非叶子结点的关键字个数=子结点个数；所有的非叶子结点可以看作索引，仅含有其子结点中的 **最大** 关键字；所有的叶子结点都在同一层，包含 **全部** 关键字及指向相应记录的指针，按大小顺序链接起来。

区别：

- B树的结点可以直接存储数据，B+树的非叶子结点仅用于索引，不存储数据
- B树更适合随机查找，B+树更适合顺序查找和范围查找
- B+树的插入和删除操作更简单，空间利用率更高

6. 线性表

定义：具有相同类型的n个数据元素的有限序列

顺序表：使用一组地址连续的存储单元来依次存储线性表中的数据元素

链表：使用一组非连续、非顺序的存储单元来存储线性表中的数据元素

区别：

- 存取方式：顺序表可以顺序存取也可以随机存取，链表只能从表头顺序存取元素
- 空间分配：顺序表需要预先分配存储空间，链表只在需要时申请分配结点空间

7. 栈与队列

栈：只能在表的后端进行插入和删除操作的线性表

队列：只能在表的前端进行删除操作，在表的后端进行插入操作的线性表

区别：

- 栈遵循 **后进先出** 原则，队列遵循 **先进先出** 原则
- 栈在遍历的过程中需要开辟临时空间保存数据，队列在遍历的过程中不影响数据结构

共享栈：两个顺序栈共享一个一维数组，栈底分别设在两端，栈顶向中间延伸。

用栈模拟队列：使用一个输入栈和一个输出栈

- 入队操作：将元素放进输入栈
- 出队操作：如果输出栈不为空，则弹出顶部元素；如果输出栈为空，则将输入栈中的所有元素依次弹出并放进输出栈
- 如果输入栈和输出栈都为空，说明队列为空

用队列模拟栈：使用一个队列

- 入栈操作：将元素入队
- 出栈操作：将除最后一个元素外的所有元素出队并重新入队，移除队首元素

8. 字符串匹配

暴力匹配：将主串与模式串的字符进行逐个比较

- 时间复杂度： $O(mn)$

KMP算法：通过预处理模式串来创建一个next数组，记录模式串中每个位置之前的子串的 **最长公共前后缀** 的长度。在匹配失败时，保持主串指针不动，回退模式串指针到有效匹配位置。

- 时间复杂度： $O(m + n)$

9. 树

特点：根结点无前驱，除根结点外的所有结点有且只有一个前驱，所有结点可以有零个或多个后继。

基本概念：

- 度：子结点的个数
- 深度：从根结点到该结点的路径长度
- 高度：从该结点到叶子结点的路径长度

满二叉树：除最后一层外，其他结点均有两棵子树。

完全二叉树：除最后一层外，其他层的结点数均达到最大值，且最后一层只在最右侧缺少结点。

堆：采用完全二叉树存储的优先队列，结点的值总是不大于或不小于其父结点的值。

二叉树遍历：

- 先序遍历：根-左-右；中序遍历：左-根-右；后序遍历：左-右-根
- 层次遍历：从上到下，从左到右

线索二叉树：如果一个结点的左指针为空，就让它左指针指向它的前驱结点，如果一个结点的右指针为空，就让它右指针指向它的后继结点。

- 优点：可以在不使用额外存储空间的情况下，实现快速的顺序访问
- 缺点：增加了插入和删除操作的复杂性

哈夫曼树：带权路径长度最短的二叉树，即权值越大的结点离根结点越近，权值越小的结点离根结点越远。

- 带权路径长度：树中所有叶子结点的带权路径长度（路径长度*结点权值）之和
- 构造：用给定的n个权值创建n棵只有一个结点的树，每次取出两棵根结点权值最小的树，组成一棵新的二叉树，其根结点的权值为两个子结点的权值之和。删除原来的两棵树，添加新得到的树，重复上述过程，直到最后只有一棵树。
- 应用：哈夫曼编码

10. 图

完全图：任意两个顶点之间都有边直接相连

连通图：无向图的任意两个顶点之间都存在路径

强连通图：有向图的任意两个顶点之间都存在双向路径

弱连通图：将有向图的有向边替换成无向边所得到的图是连通图

欧拉图：欧拉回路是指通过图中的每条 **边** 恰好一次，且开始和结束于图中同一个顶点的路径，具有欧拉回路的图称为欧拉图。

哈密顿图：哈密顿回路是指通过图中的每个 **顶点** 恰好一次，且开始和结束于图中同一个顶点的路径，具有哈密顿回路的图称为哈密顿图。

存储结构：

- 邻接矩阵：用二维数组表示图，数组角标表示顶点编号，数组元素的值表示边的有无或权值
- 邻接表：使用链表存储每个顶点的相邻顶点

图的遍历：

- 深度优先搜索：从一个起始顶点出发，沿着一条路径尽可能深入地访问顶点，直到不能继续为止，然后回溯到之前的顶点，继续尝试其他路径。使用栈来实现，时间复杂度为 $O(V + E)$ 。
- 广度优先搜索：从一个起始顶点出发，逐层访问所有邻近结点。使用队列实现，时间复杂度为 $O(V + E)$ 。

拓扑排序：将图中的所有顶点排成一个线性序列，使得对于图中的每一条有向边 $U \rightarrow V$ ，顶点U都在顶点V的前面

最短路径：

- Dijkstra算法：解决 **正权图** 的 **单源** 最短路径问题，使用 **贪心** 的思想，时间复杂度为 $O(V^2)$

过程：从未访问的结点中选择距离起始结点最近的结点，标记为已访问；对于已访问结点的所有邻居，计算起始结点通过已访问结点到达它们的距离，并更新最短路径字典，直到所有结点都被访问。

- Floyd算法：解决 **不含负权回路** 的加权图中 **任意两点** 的最短路径问题，使用 **动态规划** 的思想，时间复杂度为 $O(V^3)$

过程：创建一个距离矩阵，初始化为两个顶点之间边的权重。进行三层嵌套循环，外层循环遍历中间顶点，中层循环遍历起始顶点，内层循环遍历目标顶点。在内层循环中，计算起始顶点通过中间顶点到达目标顶点的距离，更新距离矩阵。

- Bellman-Ford算法：解决 **不含负权回路** 的加权图中 **单源** 最短路径问题，时间复杂度为 $O(VE)$

过程：创建一个距离数组，初始化为无穷大。进行V-1轮迭代，对每条边进行松弛操作，计算起始顶点通过边的一个顶点到达边的另一个顶点的距离，更新距离数组。最后再次遍历图中的所有边，如果还有边能进行松弛，则图中存在至少一个负权回路。

最小生成树：包含所有顶点，且边的总权重最小的连通子图

- Prim算法：使用 **贪心** 的思想，时间复杂度为 $O(V^2)$ ，适用于稠密图

过程：任意选择一个起始结点并加入集合，每次找出集合中的点与其他点之间权值最小的边，将对应的结点加入集合，直到所有结点都在集合中，此时由所有边构成的树即为最小生成树。

- Kruskal算法：使用 **贪心** 的思想，时间复杂度为 $O(E \log E)$ ，适用于稀疏图

过程：将所有边按权值从小到大排序，每次选择权值最小的边，如果这条边连接的两个结点不连通，则将这条边加入最小生成树，直到所有结点都连通。

11. 算法

定义：用于解决特定问题的一系列操作步骤

贪心算法：每一步都采取当前状态下的最优选择，试图达到整个问题的全局最优解。

- 应用：最短路径、最小生成树

分治算法：将原问题分解为若干个规模较小但性质相同的子问题，**递归** 解决子问题，然后合并子问题的解以解决原问题。

- 应用：快速排序、归并排序、二分查找

动态规划：将原问题分解为一系列子问题，解决子问题并 **存储** 其结果，然后根据子问题的解构建原问题的解。

- 应用：斐波那契数列、背包问题
- 要素：最优子结构、边界条件、状态转移方程

区别：分治算法的子问题之间完全独立，动态规划子问题之间可能存在重叠。

参考链接

https://blog.csdn.net/weixin_44134344/article/details/118367308

https://blog.csdn.net/weixin_51390582/article/details/133829594